🚚 Veridi Logistics — "Last Mile" Delivery Audit

AmaliTech Data Engineering Challenge

Dataset: [Olist Brazilian E-Commerce Dataset]

Notebook Structure

- **Section 0:** Setup & Data Loading
- **Story 1:** Schema Builder — Master Dataset Join
- **Story 2:** Delay Calculator — On Time vs Late Classification
- **Story 3:** Geographic Heatmap — Late Deliveries by State
- **Story 4:** Sentiment Correlation — Delays vs Review Scores
- **Bonus:** Product Category Translation (PT → EN)
- **Candidate's Choice:** Seller Risk Score Analysis

Section 0: Setup & Data Loading

> **Instructions:** Download the Olist dataset from Kaggle and place all CSV files in the **same folder** as this notebook.
>
> Direct link: https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
>
> For Google Colab, run the Kaggle download cell below.

```
# Install required libraries
!pip install kaggle plotly folium geopandas --quiet
```

```
# ────────────────────────────────────────────────────────────
# OPTION A: Download from Kaggle directly (Google Colab)
# Upload your kaggle.json API token first, then run this cell.
# ────────────────────────────────────────────────────────────
import os

# Only run if in Colab
try:
    import google.colab
    IN_COLAB = True
except ImportError:
    IN_COLAB = False

if IN_COLAB:
    from google.colab import files
    print("Upload your kaggle.json file...")
    # Uncomment the next line to upload kaggle.json
    # uploaded = files.upload()

    # Setup kaggle credentials
    # !mkdir -p ~/.kaggle
    # !cp kaggle.json ~/.kaggle/
    # !chmod 600 ~/.kaggle/kaggle.json

    # Download dataset
    # !kaggle datasets download -d olistbr/brazilian-ecommerce --unzip
    print("Kaggle download ready. Uncomment lines above after uploading kaggle.json")
else:
    print("Running locally. Ensure all CSVs are in the same folder as this notebook.")
```

```
Upload your kaggle.json file...
Kaggle download ready. Uncomment lines above after uploading kaggle.json
```

```
# ────────────────────────────────────────────────────────────
# Core Imports
# ────────────────────────────────────────────────────────────
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
import plotly.express as px
```

```
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import warnings
warnings.filterwarnings('ignore')

# Display settings
pd.set_option('display.max_columns', None)
pd.set_option('display.float_format', '{:.2f}'.format)
plt.style.use('seaborn-v0_8-whitegrid')

print("✅ Libraries loaded successfully")
```

```
✅ Libraries loaded successfully
```

```
# ────────────────────────────────────────────────────────────
# Load Raw CSVs  (relative paths — works locally and in Colab)
# ────────────────────────────────────────────────────────────
orders     = pd.read_csv('olist_orders_dataset.csv')
reviews    = pd.read_csv('olist_order_reviews_dataset.csv')
customers  = pd.read_csv('olist_customers_dataset.csv')
products   = pd.read_csv('olist_products_dataset.csv')
order_items= pd.read_csv('olist_order_items_dataset.csv')
category_translation = pd.read_csv('product_category_name_translation.csv')

print(f"orders:      {orders.shape}")
print(f"reviews:     {reviews.shape}")
print(f"customers:   {customers.shape}")
print(f"products:    {products.shape}")
print(f"order_items: {order_items.shape}")
print(f"translation: {category_translation.shape}")
```

```
orders:      (99441, 8)
reviews:     (99224, 7)
customers:   (99441, 5)
products:    (32951, 9)
order_items: (112650, 7)
translation: (71, 2)
```

```
# Quick look at the central table
print("=== Orders Table ===")
display(orders.head(3))
print("\n=== Orders dtypes ===")
print(orders.dtypes)
```

```
=== Orders Table ===
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_appr
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-07-26
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-08

```
=== Orders dtypes ===
order_id                       object
customer_id                    object
order_status                   object
order_purchase_timestamp       object
order_approved_at              object
order_delivered_carrier_date   object
order_delivered_customer_date  object
order_estimated_delivery_date  object
dtype: object
```

Story 1: The Schema Builder 🗂️

Goal: Join Orders + Reviews + Customers into a single master dataset.

Join Strategy:

- `orders` ⟵ LEFT JOIN ⟶ `reviews` on `order_id`
- `orders` ⟵ LEFT JOIN ⟶ `customers` on `customer_id`

Risk: Reviews can have multiple entries per order (1-to-many). We deduplicate by keeping the most recent review.

```
# ────────────────────────────────────────────────────────────
# Step 1a: Deduplicate reviews — keep one review per order
```

```
# ────────────────────────────────────────────────────────
print(f"Reviews before dedup: {reviews.shape[0]:,} rows")

# Convert answer timestamp to datetime for sorting
reviews['review_answer_timestamp'] = pd.to_datetime(reviews['review_answer_timestamp'])

# Sort descending so most recent review is first, then drop duplicates
reviews_dedup = (
    reviews
    .sort_values('review_answer_timestamp', ascending=False)
    .drop_duplicates(subset='order_id', keep='first')
    .reset_index(drop=True)
)
print(f"Reviews after dedup:  {reviews_dedup.shape[0]:,} rows")
print(f"Rows removed: {reviews.shape[0] - reviews_dedup.shape[0]:,}")
```

```
Reviews before dedup: 99,224 rows
Reviews after dedup:  98,673 rows
Rows removed: 551
```

```
# ────────────────────────────────────────────────────────
# Step 1b: Build the Master Dataset
# ────────────────────────────────────────────────────────

# Join 1: Orders + Reviews
master = orders.merge(
    reviews_dedup[['order_id', 'review_score', 'review_comment_title', 'review_comment_message']],
    on='order_id',
    how='left'
)

# Join 2: Master + Customers
master = master.merge(
    customers[['customer_id', 'customer_city', 'customer_state', 'customer_zip_code_prefix']],
    on='customer_id',
    how='left'
)

print(f"Orders table:  {orders.shape[0]:,} rows")
print(f"Master table:  {master.shape[0]:,} rows")
print(f"\n✅ No row duplication: {master.shape[0] == orders.shape[0]}")
print(f"\nMaster columns: {list(master.columns)}")
```

```
Orders table:  99,441 rows
Master table:  99,441 rows

✅ No row duplication: True

Master columns: ['order_id', 'customer_id', 'order_status', 'order_purchase_timestamp', 'order_approved_at', 'order_deli
```

```
# Validate: check for duplicate order_ids in master
dupe_count = master.duplicated(subset='order_id').sum()
print(f"Duplicate order_ids in master: {dupe_count}")
assert dupe_count == 0, "❌ Duplicates found! Check join logic."
print("✅ No duplicates — join is clean")

display(master.head(3))
```

```
Duplicate order_ids in master: 0
✅ No duplicates — join is clean
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_appr
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-07-26
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-08

Story 2: The "Real" Delay Calculator ⏱️

Goal: Calculate how many days off the estimated delivery was, and classify orders.

Formula:

```
Days_Difference = order_estimated_delivery_date - order_delivered_customer_date
```

- Positive → delivered BEFORE estimate (good)
- Negative → delivered AFTER estimate (bad = late)

Classification:

- `On Time` : Days_Difference >= 0
- `Late` : -5 <= Days_Difference < 0
- `Super Late` : Days_Difference < -5

```
# ────────────────────────────────────────────────────────────
# Step 2a: Parse datetime columns
# ────────────────────────────────────────────────────────────
date_cols = [
    'order_purchase_timestamp',
    'order_approved_at',
    'order_delivered_carrier_date',
    'order_delivered_customer_date',
    'order_estimated_delivery_date'
]

for col in date_cols:
    master[col] = pd.to_datetime(master[col])

print("✅ Date columns parsed")
print(master[date_cols].dtypes)
```

```
✅ Date columns parsed
order_purchase_timestamp         datetime64[ns]
order_approved_at                datetime64[ns]
order_delivered_carrier_date     datetime64[ns]
order_delivered_customer_date    datetime64[ns]
order_estimated_delivery_date    datetime64[ns]
dtype: object
```

```
# ────────────────────────────────────────────────────────────
# Step 2b: Handle missing/canceled orders
# ────────────────────────────────────────────────────────────
print("Order status distribution:")
print(master['order_status'].value_counts())

# Flag non-delivered orders separately
non_delivered_statuses = ['canceled', 'unavailable']
master['is_delivered'] = ~master['order_status'].isin(non_delivered_statuses)

# Work only with delivered orders for delay analysis
delivered = master[
    master['is_delivered'] &
    master['order_delivered_customer_date'].notna() &
    master['order_estimated_delivery_date'].notna()
].copy()

print(f"\nTotal orders:     {master.shape[0]:,}")
print(f"Delivered orders: {delivered.shape[0]:,}")
print(f"Excluded orders:  {master.shape[0] - delivered.shape[0]:,}")
```

```
Order status distribution:
order_status
delivered      96478
shipped         1107
canceled         625
unavailable      609
invoiced         314
processing       301
created            5
approved           2
Name: count, dtype: int64

Total orders:     99,441
Delivered orders: 96,470
Excluded orders:  2,971
```

```
# ────────────────────────────────────────────────────────────
# Step 2c: Calculate Days_Difference and classify
# ────────────────────────────────────────────────────────────

# Days_Difference: positive = early, negative = late
delivered['Days_Difference'] = (
```

```
    delivered['order_estimated_delivery_date'] - delivered['order_delivered_customer_date']
).dt.days

def classify_delivery(days):
    if days >= 0:
        return 'On Time'
    elif days >= -5:
        return 'Late'
    else:
        return 'Super Late'

delivered['delivery_status'] = delivered['Days_Difference'].apply(classify_delivery)

# Summary
status_counts = delivered['delivery_status'].value_counts()
status_pct    = delivered['delivery_status'].value_counts(normalize=True) * 100

print("=== Delivery Status Summary ===")
summary = pd.DataFrame({'Count': status_counts, 'Percentage': status_pct.round(1)})
display(summary)

print(f"\nAverage Days Difference: {delivered['Days_Difference'].mean():.1f} days")
print(f"Median Days Difference:  {delivered['Days_Difference'].median():.1f} days")
```

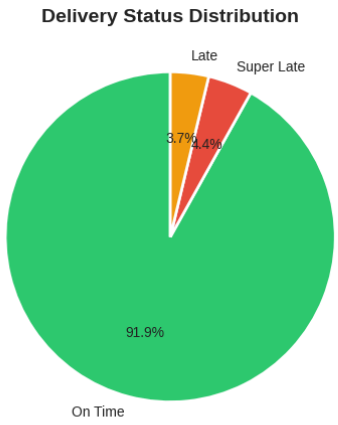
```
=== Delivery Status Summary ===
```

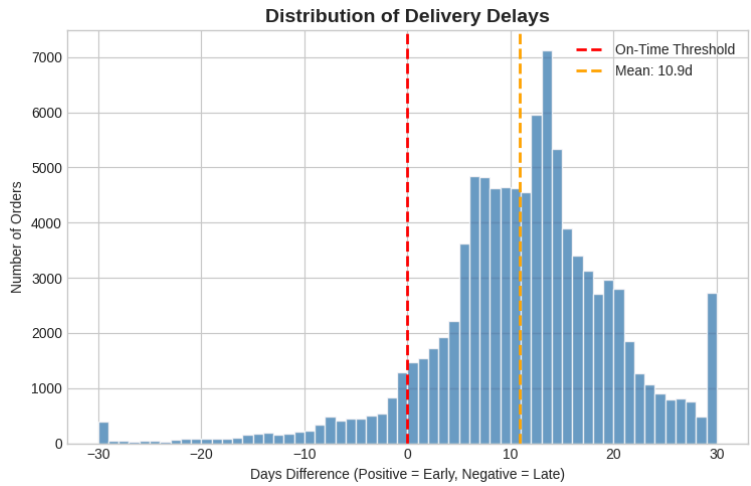
delivery_status	Count	Percentage
On Time	88644	91.90
Super Late	4211	4.40
Late	3615	3.70

```
Average Days Difference: 10.9 days
Median Days Difference:  11.0 days
```

Next steps: Generate code with summary New interactive sheet

```
# ────────────────────────────────────────────────────────────
# Story 2 Visualization: Delivery Status Breakdown
# ────────────────────────────────────────────────────────────
colors = {'On Time': '#2ecc71', 'Late': '#f39c12', 'Super Late': '#e74c3c'}

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Pie chart
ax1 = axes[0]
status_counts.plot(
    kind='pie', ax=ax1,
    colors=[colors[s] for s in status_counts.index],
    autopct='%1.1f%%', startangle=90,
    wedgeprops={'edgecolor': 'white', 'linewidth': 2}
)
ax1.set_title('Delivery Status Distribution', fontsize=14, fontweight='bold')
ax1.set_ylabel('')

# Histogram of Days_Difference
ax2 = axes[1]
ax2.hist(delivered['Days_Difference'].clip(-30, 30), bins=60,
         color='steelblue', edgecolor='white', alpha=0.8)
ax2.axvline(0, color='red', linestyle='--', linewidth=2, label='On-Time Threshold')
ax2.axvline(delivered['Days_Difference'].mean(), color='orange',
            linestyle='--', linewidth=2, label=f"Mean: {delivered['Days_Difference'].mean():.1f}d")
ax2.set_xlabel('Days Difference (Positive = Early, Negative = Late)')
ax2.set_ylabel('Number of Orders')
ax2.set_title('Distribution of Delivery Delays', fontsize=14, fontweight='bold')
ax2.legend()

plt.tight_layout()
plt.savefig('story2_delivery_status.png', dpi=150, bbox_inches='tight')
plt.show()
print("✅ Chart saved as story2_delivery_status.png")
```





✅ Chart saved as story2_delivery_status.png

Story 3: The Geographic Heatmap 🗺️

Goal: Which Brazilian states have the highest % of late deliveries?

```
# ────────────────────────────────────────────────────────────
# Step 3a: Calculate late % per state
# ────────────────────────────────────────────────────────────
delivered['is_late'] = delivered['delivery_status'].isin(['Late', 'Super Late'])

state_stats = delivered.groupby('customer_state').agg(
    total_orders   = ('order_id', 'count'),
    late_orders    = ('is_late', 'sum'),
    avg_delay_days = ('Days_Difference', 'mean'),
    avg_review     = ('review_score', 'mean')
).reset_index()

state_stats['late_pct'] = (state_stats['late_orders'] / state_stats['total_orders'] * 100).round(1)
state_stats = state_stats.sort_values('late_pct', ascending=False)

print("=== Top 10 States by Late Delivery % ===")
display(state_stats.head(10))

print("\n=== Bottom 5 States (Best Performers) ===")
display(state_stats.tail(5))
```

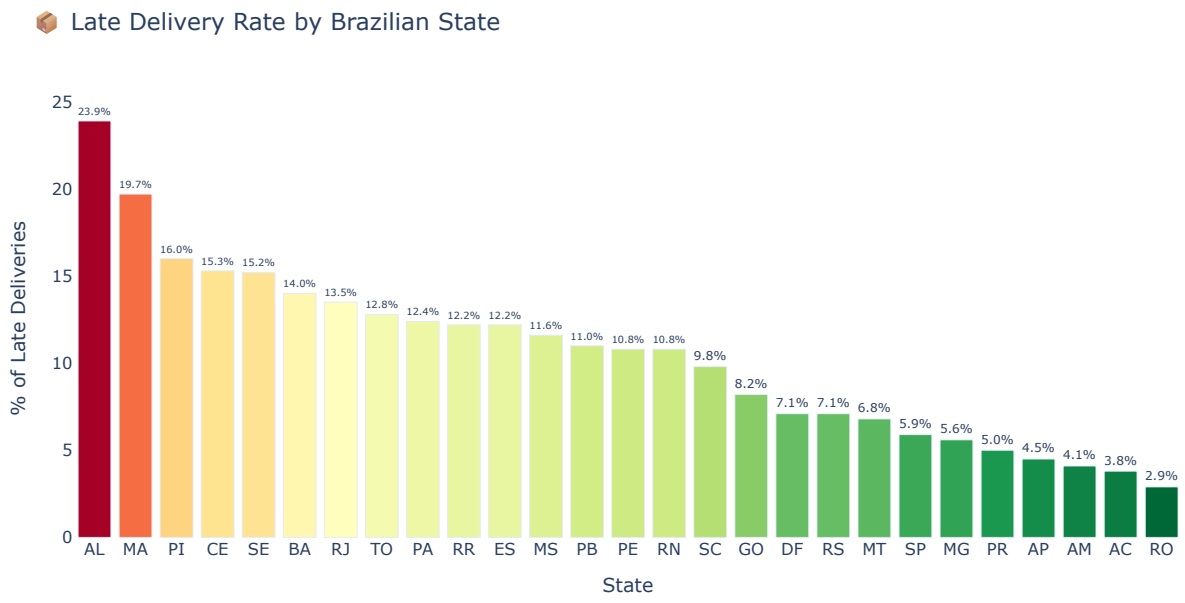
```
=== Top 10 States by Late Delivery % ===
```

	customer_state	total_orders	late_orders	avg_delay_days	avg_review	late_pct
1	AL	397	95	7.71	3.85	23.90
9	MA	717	141	8.57	3.83	19.70
16	PI	476	76	10.31	3.99	16.00
5	CE	1279	196	9.80	3.94	15.30
24	SE	335	51	9.02	3.91	15.20
4	BA	3256	457	9.79	3.93	14.00
18	RJ	12350	1664	10.76	3.97	13.50
26	TO	274	35	11.13	4.15	12.80
13	PA	946	117	13.07	3.91	12.40
21	RR	41	5	16.29	3.90	12.20

```
=== Bottom 5 States (Best Performers) ===
```

	customer_state	total_orders	late_orders	avg_delay_days	avg_review	late_pct
17	PR	4923	246	12.31	4.24	5.00
3	AP	67	3	18.69	4.24	4.50
2	AM	145	6	18.57	4.24	4.10
0	AC	80	3	19.73	4.09	3.80
20	RO	243	7	19.10	4.17	2.90

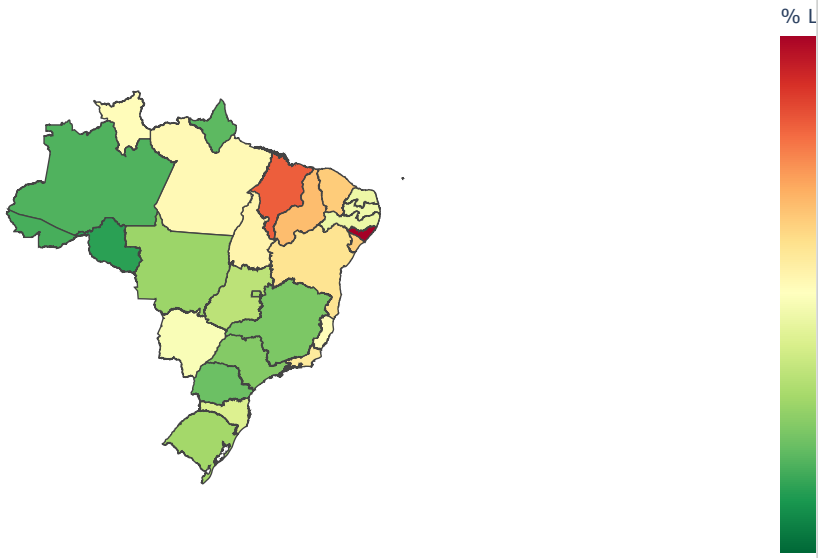
```
# ────────────────────────────────────────────────────────────
# Story 3: Bar Chart — Late % by State
# ────────────────────────────────────────────────────────────
fig = px.bar(
    state_stats,
    x='customer_state',
    y='late_pct',
    color='late_pct',
    color_continuous_scale='RdYlGn_r',
    text='late_pct',
    labels={
        'customer_state': 'Brazilian State',
        'late_pct': '% Late Deliveries',
    },
    title='📦 Late Delivery Rate by Brazilian State',
    hover_data=['total_orders', 'avg_review']
)
fig.update_traces(texttemplate='%{text:.1f}%', textposition='outside')
fig.update_layout(
    height=500,
    xaxis_title='State',
    yaxis_title='% of Late Deliveries',
    coloraxis_showscale=False,
    plot_bgcolor='white'
)
fig.show()
fig.write_html('story3_late_by_state.html')
print("✅ Chart saved as story3_late_by_state.html")
```




```
✅ Chart saved as story3_late_by_state.html
```

```
# ────────────────────────────────────────────────────────────
# Story 3: Choropleth Map — Brazil States
# ────────────────────────────────────────────────────────────
# Brazil state GeoJSON (public domain)
import urllib.request, json

geojson_url = "https://raw.githubusercontent.com/codeforamerica/click_that_hood/master/public/data/brazil-states.geojson

try:
    with urllib.request.urlopen(geojson_url) as response:
        brazil_geo = json.load(response)

    # Match state abbreviations: GeoJSON uses 'sigla' property
    fig_map = px.choropleth(
        state_stats,
        geojson=brazil_geo,
        locations='customer_state',
        featureidkey='properties.sigla',
        color='late_pct',
        color_continuous_scale='RdYlGn_r',
        range_color=[0, state_stats['late_pct'].max()],
        scope='south america',
        labels={'late_pct': '% Late'},
        title='🗺️ Late Delivery Heatmap — Brazil',
        hover_data=['total_orders', 'avg_review', 'avg_delay_days']
    )
    fig_map.update_geos(fitbounds='locations', visible=False)
    fig_map.update_layout(height=600)
    fig_map.show()
    fig_map.write_html('story3_brazil_heatmap.html')
    print("✅ Map saved as story3_brazil_heatmap.html")
except Exception as e:
    print(f"Could not load GeoJSON (check internet): {e}")
    print("Falling back to bar chart above.")
```

🗺️ Late Delivery Heatmap — Brazil



✅ Map saved as story3_brazil_heatmap.html

Story 4: The Sentiment Correlation

Goal: Prove that late deliveries cause bad reviews.

```
# ────────────────────────────────────────────────────────────
# Step 4a: Average review score by delivery status
# ────────────────────────────────────────────────────────────
sentiment = delivered.groupby('delivery_status').agg(
    avg_review_score = ('review_score', 'mean'),
    order_count      = ('order_id', 'count')
).reset_index()

# Order for display
status_order = ['On Time', 'Late', 'Super Late']
sentiment['delivery_status'] = pd.Categorical(sentiment['delivery_status'], categories=status_order, ordered=True)
sentiment = sentiment.sort_values('delivery_status')

print("=== Avg Review Score by Delivery Status ===")
display(sentiment)
```

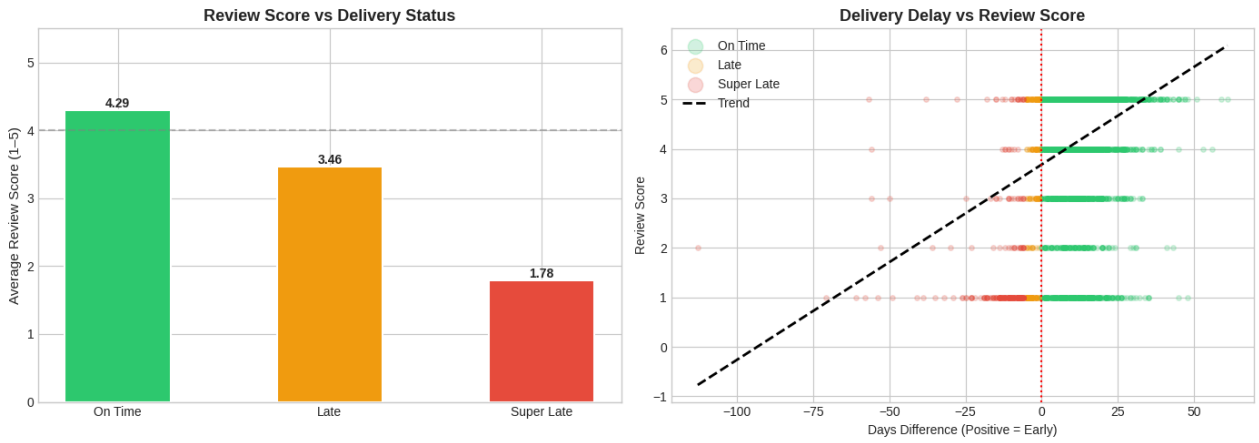
```
=== Avg Review Score by Delivery Status ===
```

	delivery_status	avg_review_score	order_count
1	On Time	4.29	88644
0	Late	3.46	3615
2	Super Late	1.78	4211

Next steps: Generate code with sentiment | New interactive sheet

```
# ────────────────────────────────────────────────────────────
# Story 4 Chart 1: Avg Review Score by Delivery Status
# ────────────────────────────────────────────────────────────
palette = {'On Time': '#2ecc71', 'Late': '#f39c12', 'Super Late': '#e74c3c'}

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Bar chart
ax1 = axes[0]
bars = ax1.bar(
    sentiment['delivery_status'],
```

```
    sentiment['avg_review_score'],
    color=[palette[s] for s in sentiment['delivery_status']],
    edgecolor='white', linewidth=1.5, width=0.5
)
ax1.set_ylim(0, 5.5)
ax1.set_ylabel('Average Review Score (1–5)', fontsize=11)
ax1.set_title('Review Score vs Delivery Status', fontsize=13, fontweight='bold')
ax1.axhline(y=4, color='gray', linestyle='--', alpha=0.5, label='Score = 4')
for bar, val in zip(bars, sentiment['avg_review_score']):
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.05,
             f'{val:.2f}', ha='center', fontweight='bold')

# Scatter: Days_Difference vs Review Score (sampled for clarity)
ax2 = axes[1]
sample = delivered[delivered['review_score'].notna()].sample(min(5000, len(delivered)), random_state=42)
for status, color in palette.items():
    subset = sample[sample['delivery_status'] == status]
    ax2.scatter(subset['Days_Difference'], subset['review_score'],
                alpha=0.2, c=color, label=status, s=15)

# Trend line
z = np.polyfit(sample['Days_Difference'].fillna(0), sample['review_score'].fillna(0), 1)
p = np.poly1d(z)
x_line = np.linspace(sample['Days_Difference'].min(), sample['Days_Difference'].max(), 100)
ax2.plot(x_line, p(x_line), 'k--', linewidth=2, label='Trend')
ax2.axvline(0, color='red', linestyle=':', linewidth=1.5)
ax2.set_xlabel('Days Difference (Positive = Early)')
ax2.set_ylabel('Review Score')
ax2.set_title('Delivery Delay vs Review Score', fontsize=13, fontweight='bold')
ax2.legend(markerscale=3)

plt.tight_layout()
plt.savefig('story4_sentiment_correlation.png', dpi=150, bbox_inches='tight')
plt.show()
print("✅ Chart saved as story4_sentiment_correlation.png")
```




```
✅ Chart saved as story4_sentiment_correlation.png
```

```
# ────────────────────────────────────────────────────────────
# Step 4b: Correlation coefficient
# ────────────────────────────────────────────────────────────
corr_data = delivered[['Days_Difference', 'review_score']].dropna()
correlation = corr_data['Days_Difference'].corr(corr_data['review_score'])
print(f"Pearson Correlation (Days_Difference vs Review Score): {correlation:.3f}")

if correlation > 0.3:
    print("✅ Moderate to strong positive correlation — earlier delivery = better reviews")
elif correlation > 0.1:
    print("⚠️  Weak positive correlation exists")
else:
    print("📊 Correlation is weak — other factors also drive review scores")

# Review score distribution by status
print("\n=== Review Score Distribution by Status ===")
display(
    delivered.groupby('delivery_status')['review_score']
    .value_counts(normalize=True)
    .mul(100)
    .round(1)
    .rename('percentage')
    .reset_index()
    .pivot(index='review_score', columns='delivery_status', values='percentage')
    .fillna(0)
```

```
    .fillna(0)
)
```

```
Pearson Correlation (Days_Difference vs Review Score): 0.267
⚠️  Weak positive correlation exists

=== Review Score Distribution by Status ===
```

delivery_status	Late	On Time	Super Late
review_score			
1.00	22.00	6.60	67.30
2.00	6.00	2.60	9.50
3.00	14.00	8.00	9.10
4.00	20.00	20.40	5.60
5.00	38.00	62.40	8.50

Bonus: Product Category Translation 🌍

Goal: Translate Portuguese product categories to English for the dashboard.

```
# ────────────────────────────────────────────────────────────
# Join products → translation → order_items → delivered
# ────────────────────────────────────────────────────────────
print("Translation file preview:")
display(category_translation.head())

# Merge products with translation
products_en = products.merge(category_translation, on='product_category_name', how='left')
products_en['category_en'] = products_en['product_category_name_english'].fillna('unknown')

# Add product to order items
items_with_category = order_items.merge(
    products_en[['product_id', 'category_en']],
    on='product_id', how='left'
)

# One category per order (take first item's category)
order_category = (
    items_with_category
    .groupby('order_id')['category_en']
    .first()
    .reset_index()
)

# Merge into delivered
delivered = delivered.merge(order_category, on='order_id', how='left')

print(f"\nTop 10 product categories:")
display(delivered['category_en'].value_counts().head(10))
```

```
Translation file preview:
```

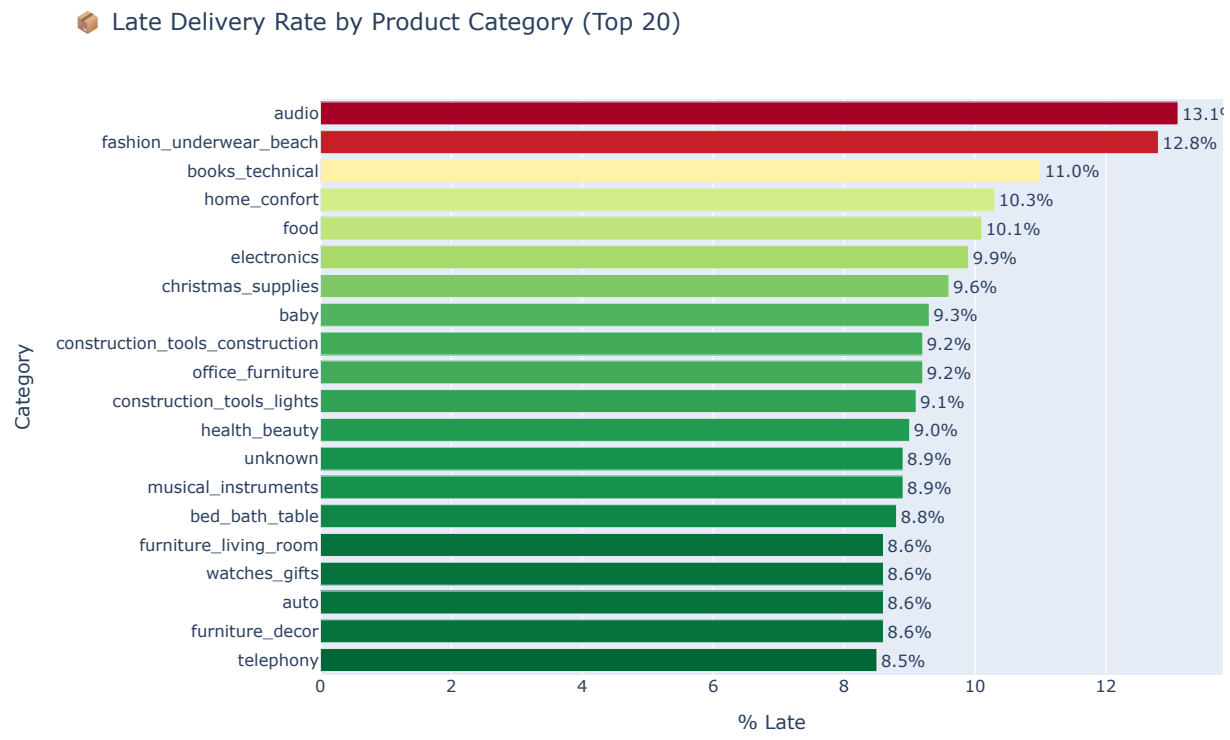
	product_category_name	product_category_name_english
0	beleza_saude	health_beauty
1	informatica_acessorios	computers_accessories
2	automotivo	auto
3	cama_mesa_banho	bed_bath_table
4	moveis_decoracao	furniture_decor

```
Top 10 product categories:
```

category_en	count
bed_bath_table	9167
health_beauty	8608
sports_leisure	7490
computers_accessories	6500
furniture_decor	6213
housewares	5688
watches_gifts	5470
telephony	4076
auto	3792
toys	3778

dtype: int64

```
# ────────────────────────────────────────────────────────────
# Bonus Chart: Late Rate by Product Category
# ────────────────────────────────────────────────────────────
cat_stats = delivered.groupby('category_en').agg(
    total_orders = ('order_id', 'count'),
    late_orders  = ('is_late', 'sum'),
    avg_review   = ('review_score', 'mean')
).reset_index()

cat_stats['late_pct'] = (cat_stats['late_orders'] / cat_stats['total_orders'] * 100).round(1)

# Filter categories with at least 100 orders
cat_stats_filtered = cat_stats[cat_stats['total_orders'] >= 100].sort_values('late_pct', ascending=False).head(20)

fig = px.bar(
    cat_stats_filtered,
    y='category_en', x='late_pct',
    orientation='h',
    color='late_pct',
    color_continuous_scale='RdYlGn_r',
    title='📦 Late Delivery Rate by Product Category (Top 20)',
    labels={'category_en': 'Category', 'late_pct': '% Late'},
    text='late_pct'
)
fig.update_traces(texttemplate='%{text:.1f}%', textposition='outside')
fig.update_layout(height=600, yaxis={'categoryorder': 'total ascending'}, coloraxis_showscale=False)
fig.show()
fig.write_html('bonus_category_late_rate.html')
print("✅ Chart saved")
```



✅ Chart saved

Candidate's Choice: Seller Risk Score 🎯

What it is: A composite risk score per seller based on:

- Their late delivery rate
- Their average review score
- Their volume of super-late orders

Why it matters to the business: The CEO's problem might not be purely a regional logistics issue — specific sellers may be setting unrealistic ETAs, or using slow carriers. Identifying high-risk sellers allows Veridi to take targeted action (retraining, penalty, removal) rather than overhauling the entire logistics network.

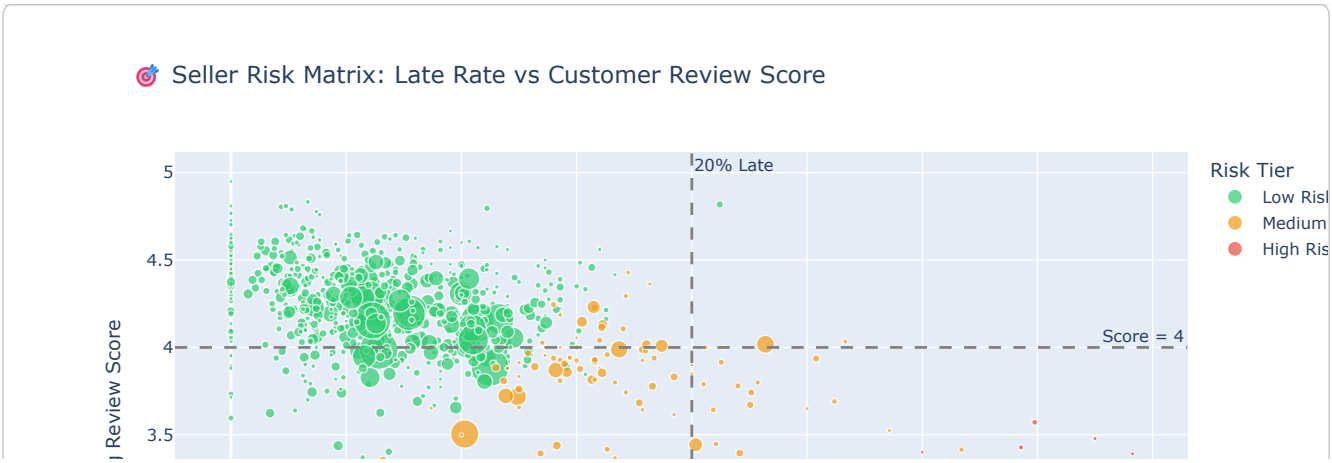
```
# ────────────────────────────────────────────────────────────
# Seller Risk Score
# ────────────────────────────────────────────────────────────
# Join sellers to delivered orders via order_items
delivered_with_seller = delivered.merge(
    order_items[['order_id', 'seller_id']].drop_duplicates(subset='order_id'),
    on='order_id', how='left'
)

seller_stats = delivered_with_seller.groupby('seller_id').agg(
    total_orders    = ('order_id', 'count'),
    late_orders     = ('is_late', 'sum'),
    super_late      = ('delivery_status', lambda x: (x == 'Super Late').sum()),
    avg_review      = ('review_score', 'mean'),
    avg_delay_days  = ('Days_Difference', 'mean')
).reset_index()

# Only sellers with >= 20 orders for statistical reliability
seller_stats = seller_stats[seller_stats['total_orders'] >= 20].copy()

seller_stats['late_pct']       = seller_stats['late_orders'] / seller_stats['total_orders'] * 100
seller_stats['super_late_pct'] = seller_stats['super_late'] / seller_stats['total_orders'] * 100

# Normalize components to 0-100 scale
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range=(0, 100))

seller_stats['late_score']       = scaler.fit_transform(seller_stats[['late_pct']])
seller_stats['review_penalty']   = scaler.fit_transform((5 - seller_stats[['avg_review']]))
seller_stats['super_late_score'] = scaler.fit_transform(seller_stats[['super_late_pct']])
```

```
# Composite Risk Score (weighted)
seller_stats['risk_score'] = (
    0.40 * seller_stats['late_score'] +
    0.35 * seller_stats['review_penalty'] +
    0.25 * seller_stats['super_late_score']
).round(1)

# Categorize risk
seller_stats['risk_tier'] = pd.cut(
    seller_stats['risk_score'],
    bins=[0, 33, 66, 100],
    labels=['Low Risk', 'Medium Risk', 'High Risk']
)

print(f"Sellers analyzed: {len(seller_stats)}")
print("\nRisk tier distribution:")
print(seller_stats['risk_tier'].value_counts())
print("\nTop 10 Highest Risk Sellers:")
display(
    seller_stats.nlargest(10, 'risk_score')[
        ['seller_id', 'total_orders', 'late_pct', 'avg_review', 'risk_score', 'risk_tier']
    ].round(2)
)
```

```
Sellers analyzed: 794

Risk tier distribution:
risk_tier
Low Risk       686
Medium Risk    101
High Risk        6
Name: count, dtype: int64

Top 10 Highest Risk Sellers:
```

	seller_id	total_orders	late_pct	avg_review	risk_score	risk_tier
2002	ad781527c93d00d89a11eecd9dcad7c1	35	34.29	3.43	81.10	High Risk
994	54965bbe3e4f07ae045b90b0b8541f52	72	30.56	3.18	78.90	High Risk
2756	ede0c03645598cdfc63ca8237acbe73d	43	34.88	3.57	77.00	High Risk
2863	f76a3b1349b6df1ee875d1f3fa4340f0	23	39.13	3.39	75.40	High Risk
56	054694fa03fe82cec4b7551487331d74	20	30.00	3.40	72.00	High Risk
1536	821fb029fc6e495ca4f08a35d51e53a5	24	37.50	3.48	72.00	High Risk
474	2a1348e9addc1af5aaa619b1a3679d6b	48	27.08	3.17	65.50	Medium Risk
909	4e5725ba188db8252977a4f0227bd462	21	28.57	3.52	64.20	Medium Risk
1920	a49928bcdf77c55c6d6e05e09a9b4ca5	94	26.60	3.06	64.20	Medium Risk
321	1ca7077d890b907f89be8c954a02686a	108	22.22	2.39	63.60	Medium Risk

```
# ────────────────────────────────────────────────────────────
# Seller Risk Chart: Scatter plot
# ────────────────────────────────────────────────────────────
tier_colors = {'Low Risk': '#2ecc71', 'Medium Risk': '#f39c12', 'High Risk': '#e74c3c'}

fig = px.scatter(
    seller_stats,
    x='late_pct',
    y='avg_review',
    color='risk_tier',
    color_discrete_map=tier_colors,
    size='total_orders',
    hover_data=['seller_id', 'total_orders', 'risk_score', 'super_late_pct'],
    title='🎯 Seller Risk Matrix: Late Rate vs Customer Review Score',
    labels={
        'late_pct': '% Late Deliveries',
        'avg_review': 'Avg Review Score',
        'risk_tier': 'Risk Tier'
    }
)
fig.add_hline(y=4, line_dash='dash', line_color='gray', annotation_text='Score = 4')
fig.add_vline(x=20, line_dash='dash', line_color='gray', annotation_text='20% Late')
fig.update_layout(height=550)
fig.show()
fig.write_html('choice_seller_risk.html')
print("✅ Seller risk chart saved")
```


🎯 Seller Risk Matrix: Late Rate vs Customer Review Score
20% Late
Score = 4
5
4.5
4
3.5
Review Score
Risk Tier
Low Risl
Medium
High Ris